

python : Les bases

Table des matières

1. Prés-requis
2. Variables
3. Interaction avec la Console
4. Opérateurs
5. Structures Conditionnelles
6. Structures Itératives
7. Collections
8. Gestion des Erreurs
9. Fonctions et procédures

1. Prés-requis

- Introduction à Python
- Installation de Python
- Éditeurs de Code

Introduction à python

Python est un langage de programmation polyvalent et facile à apprendre, largement utilisé dans divers domaines tels que le développement web, l'analyse de données ou l'intelligence artificielle.

Sa syntaxe claire et concise en fait un excellent choix pour les débutants en programmation ainsi que pour les professionnels expérimentés. En raison de sa popularité croissante et de sa communauté active, Python est continuellement mis à jour avec de nouvelles fonctionnalités et améliorations.

Installation de Python

Installer Python est une étape essentielle pour commencer à programmer dans ce langage. Python peut être installé sur différentes plateformes telles que Windows, macOS et Linux.

Il existe plusieurs méthodes d'installation, y compris l'utilisation de gestionnaires de paquets comme apt pour Linux, brew pour macOS, ou en téléchargeant directement depuis le site officiel de Python.

Il est recommandé d'utiliser la dernière version stable de Python pour bénéficier des dernières fonctionnalités et corrections de bogues.

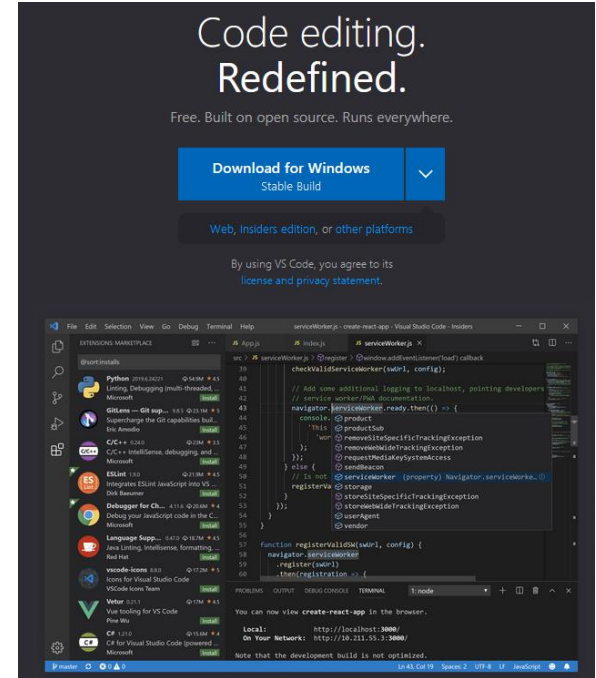
<https://www.python.org/downloads/>

Python version	Maintenance status	First released	End of support	Release schedule
3.13	prerelease	2024-10-01 (planned)	2029-10	PEP 719
3.12	bugfix	2023-10-02	2028-10	PEP 693
3.11	bugfix	2022-10-24	2027-10	PEP 664
3.10	security	2021-10-04	2026-10	PEP 619
3.9	security	2020-10-05	2025-10	PEP 596
3.8	security	2019-10-14	2024-10	PEP 569

Éditeurs de code

Nous allons utiliser Visual Studio Code (VSCode) pour notre cours, car il offre une prise en charge avancée pour le développement Python, facilitant ainsi la rédaction, le débogage et la gestion de projets Python.

Parmi les éditeurs les plus utilisés, on trouve également PyCharm, Sublime Text, Atom, et Vim, chacun offrant ses propres fonctionnalités et extensions pour améliorer notre productivité.



<https://code.visualstudio.com/>

2. Variables

- Introduction aux Variables
- Convention et bonnes pratiques
- Typage dynamique
- Utilisations
- Types de données
- Exemples d'utilisation
- Conversions

Introduction aux variables

En programmation, une variable est un élément fondamental permettant de stocker et manipuler des données dans un programme informatique. Une variable est constituée de plusieurs éléments :

- **Nom de la variable** : Il s'agit du nom que vous donnez à la variable pour l'identifier dans votre programme.
- **Valeur** : C'est la donnée stockée dans la variable.
- **Type de données** : Chaque variable a un type de données qui détermine le genre de données qu'elle peut stocker et les opérations qui peuvent être effectuées sur cette donnée.
- **Emplacement en mémoire** : Chaque variable est associée à une adresse mémoire où sa valeur est stockée.

Introduction aux variables

Contrairement à certains autres langages, Python ne nécessite pas de déclaration de type explicite pour les variables.

Cela signifie que nous pourrions simplement déclarer une variable et lui assigner une valeur, sans avoir à spécifier le type de données.

```
1  name = "John"  
2  age = 30
```

Conventions et bonnes pratiques

Il est important de suivre des conventions et des bonnes pratiques pour nommer les variables afin d'écrire un code clair, compréhensible et cohérent.

- **Les caractères spéciaux sont interdits :** (Lettres accentuées, %, \$, ect...)
- **Utiliser des noms significatifs :** Les noms de variables doivent être descriptifs et refléter le but ou le contenu de la variable. Évitez les noms génériques comme "x" ou "data", et privilégiez des noms explicites comme "compteur_de_clients" ou "liste_des_utilisateurs".
- **Utiliser la casse snake_case :** En Python, il est courant d'utiliser la convention **snake_case** pour nommer les variables, où les mots sont séparés par des underscores. Par exemple, "nom_utilisateur" ou "total_ventes".
- **Utiliser des noms de variables en minuscules :** Les noms de variables doivent être en minuscules pour distinguer visuellement les variables des classes et des fonctions, qui suivent généralement la convention **PascalCase** ou **camelCase**.

Conventions et bonnes pratiques

- **Suivre les conventions de nommage spécifiques au projet** : Si vous travaillez sur un projet avec des conventions de nommage spécifiques, assurez-vous de les suivre pour maintenir la cohérence du code au sein de l'équipe de développement.
- **Éviter les noms de variables réservés** : Évitez d'utiliser des noms de variables qui sont déjà des mots-clés ou des fonctions intégrées en Python, comme :

and	as	assert	break	class	continue	def
del	elif	else	except	False	finally	for
from	global	if	import	in	is	lambda
None	nonlocal	not	or	pass	raise	return
True	try	while	with	yield		

Typage Dynamique

Python est un langage à typage dynamique, ce qui signifie que le type d'une variable est déterminé automatiquement par Python lors de l'exécution du programme. Par conséquent, nous pourrions réassigner une variable à une valeur d'un type de données différent sans aucun problème.

Par exemple, nous pourrions initialement déclarer `x = 10`, puis réassigner `x = "hello"` sans rencontrer d'erreur.

```
1  x = 10
2  x = "Hello"
```

⚠ Il est fortement déconseillé de changer le type de variable au cours du programme

Typage Dynamique

La fonction `type()` est utilisée pour déterminer le type de données d'une variable ou d'une valeur. En l'appelant avec une variable ou une valeur en argument, elle renvoie le type de données associé à cette variable ou à cette valeur.

Cela nous permet de connaître le type exact des données avec lesquelles nous travaillons, ce qui est essentiel pour prendre des décisions appropriées dans le code, telles que des conversions de types ou des opérations spécifiques selon le type.

```
1  x = 5
2  print(type(x))  # Affiche <class 'int'>
3
4
5  x = "Hello"
6  if type(x) == str:
7      print("x est une chaîne de caractères.")
```

Utilisation : Déclaration et affectation

nous déclarons et utilisons des variables de manière simple et intuitive. Nous attribuons une valeur à un nom de variable pour la déclarer. Par exemple, dans le code `name = "John"`, nous définissons la variable `name` avec la valeur `"John"`, et avec `age = 30`, nous attribuons la valeur `30` à la variable `age`. Ces variables peuvent être utilisées dans des expressions et des instructions pour effectuer des opérations.

En utilisant des f-strings, nous pouvons combiner ces variables pour créer un message personnalisé.

```
1  name = "John"
2  age = 30
3  message = f"Bonjour, je m'appelle {name} et j'ai {age} ans."
4  print(message)
```

Utilisation : affectation multiple

L'affectation multiple en Python permet d'assigner une même valeur à plusieurs variables en une seule instruction, ce qui rend le code plus concis et lisible. Cette technique consiste à utiliser l'opérateur d'affectation pour assigner une valeur à plusieurs variables simultanément.

```
1  # Assigner une même valeur à plusieurs variables
2  x = y = z = 0
3  print("Assignment d'une même valeur - x :", x, "y :", y, "z :", z)
```

Utilisation : affectation parallèle

L'affectation parallèle est une technique puissante permettant d'assigner des valeurs à plusieurs variables simultanément en une seule instruction. Elle est réalisée en attribuant les valeurs de droite à gauche, permettant ainsi d'échanger facilement les valeurs de deux variables sans nécessiter de variable temporaire.

```
1  # Définir des valeurs
2  a = 5
3  b = 10
4
5  # Affectation multiple pour échanger les valeurs de a et b
6  a, b = b, a
```

Types de données

En Python, nous avons accès à une diversité de données, chacune ayant ses propres caractéristiques et usages distincts.

Cette variété nous permet de manipuler efficacement les données dans nos programmes.

En tirant parti de ces fonctionnalités, nous pourrons créer des applications puissantes et flexibles.

Exemples pratiques

Type de Données	Description	Exemple
int	Nombre entier sans décimales	5, -3, 100
float	Nombre à virgule flottante	3.14, -0.001, 2.5
str	Chaîne de caractères	"Bonjour", 'Python', "123"
bool	Valeur de vérité (True ou False)	True, False
list	Collection ordonnée modifiable	[1, 2, 3], ["a", "b", "c"]
tuple	Collection ordonnée immuable	(1, 2, 3), ("a", "b", "c")
set	Collection non ordonnée d'éléments uniques	{1, 2, 3}, {"a", "b", "c"}
dict	Collection de paires clé-valeur	{"nom": "John", "âge": 30}

Conversion

Les conversions en Python sont des mécanismes qui permettent de transformer les données d'un type à un autre. Elles jouent un rôle essentiel dans le traitement des données en offrant la capacité de changer entre les types de données.

Fonction	Description
<code>int()</code>	Convertit une valeur en un entier.
<code>float()</code>	Convertit une valeur en un nombre à virgule flottante.
<code>str()</code>	Convertit une valeur en une chaîne de caractères.
<code>list()</code>	Convertit une séquence en une liste.
<code>tuple()</code>	Convertit une séquence en un tuple.
<code>dict()</code>	Convertit une séquence de paires clé-valeur en un dictionnaire.
<code>bool()</code>	Convertit une valeur en un booléen.
<code>eval()</code>	Évalue une expression Python passée en tant que chaîne de caractères et retourne le résultat.

Exercices

- Trouvez une méthode permettant d'inverser le contenu de deux variables (du même type évidemment). Si $A = 5$ et $B = 7$, après ce traitement, $A = 7$ et $B = 5$.

3. Interaction avec la console

- Utilités
- Fonction `input()`
- Fonction `print()`
- Astuces pour l'Interaction avec la Console

Utilités

L'interaction avec la console dans le développement de logiciels est un outil indispensable offrant une multitude d'avantages.

Tout d'abord, elle nous permet de tester rapidement des fragments de code et d'expérimenter différentes fonctionnalités sans la nécessité de créer un programme complet.

Ensuite, elle facilite le processus de débogage en fournissant des retours immédiats sur l'exécution du code, permettant ainsi d'identifier et de corriger rapidement les erreurs.

Enfin, elle constitue également un outil d'apprentissage efficace pour les débutants en programmation, leur permettant d'expérimenter directement avec le code et de comprendre son fonctionnement de manière interactive.

La fonction input()

La fonction `input()` permet à nos programmes d'interagir avec les utilisateurs en leur demandant de saisir des données directement depuis la console.

Lorsque la fonction `input()` est appelée, le programme se met en pause et attend que l'utilisateur entre une valeur. Cette valeur est ensuite renvoyée sous forme de chaîne de caractères, que nous pouvons ensuite traiter dans notre programme.

```
1  # Demander à l'utilisateur son nom
2  nom = input("Entrez votre nom : ")
```

La fonction print()

La fonction print() nous permet de communiquer avec les utilisateurs en affichant des messages, des résultats de calculs ou d'autres informations pertinentes.

L'utilisation de la fonction print() est simple : nous fournissons les éléments que nous voulons afficher, séparés par des virgules, et Python les affiche dans la console.

```
4  # Afficher un message de salutation  
5  print("Bonjour,", nom, "! Bienvenue dans notre programme.")
```

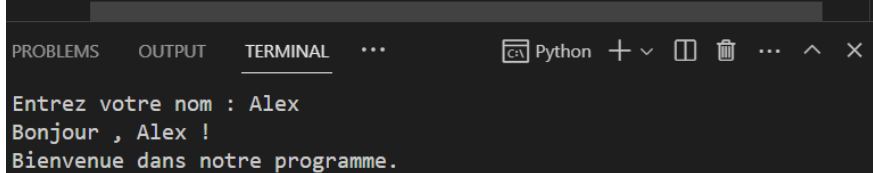
Astuces pour l'interaction avec la console

L'astuce du caractère de contrôle `\n` est une méthode simple mais efficace pour améliorer la lisibilité des messages affichés dans la console en créant des retours à la ligne.

En plaçant `\n` à la fin d'une chaîne de caractères utilisés avec la fonction `print()`, nous pouvons créer un espace visuel entre les différentes parties du message.

Par exemple, en utilisant `print("Entrez votre nom :\n")`, nous introduisons un retour à la ligne après le message d'instruction, ce qui sépare clairement les instructions des données saisies par l'utilisateur.

```
12 # Demander à l'utilisateur son nom
13 nom = input("Entrez votre nom : ")
14
15 # Afficher un message de salutation
16 print("Bonjour ,", nom, "! \nBienvenue dans notre programme.")
```



PROBLEMS OUTPUT TERMINAL ... Python + - [] [X] ... ^ X

Entrez votre nom : Alex
Bonjour , Alex !
Bienvenue dans notre programme.

Astuces pour l'interaction avec la console

Dans Python, la fonction `print()` offre deux paramètres facultatifs, `sep` et `end`, qui permettent de personnaliser la sortie lors de l'impression de plusieurs éléments.

Le paramètre `sep` permet de spécifier le séparateur entre les éléments à imprimer, tandis que le paramètre `end` permet de définir la chaîne à ajouter à la fin de l'impression.

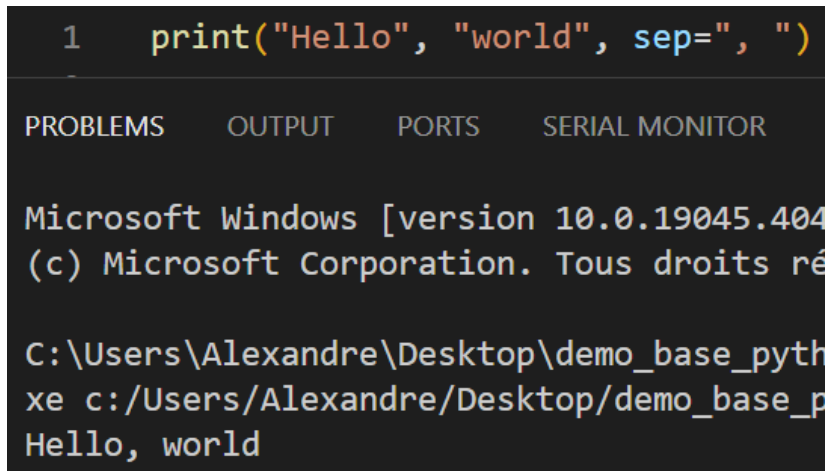
Ces fonctionnalités offrent un contrôle précis sur le formatage de la sortie, rendant ainsi la fonction `print()` encore plus flexible

Astuces pour l'interaction avec la console

sep :

- Permet de spécifier le séparateur entre les éléments à imprimer.
- Par défaut, le séparateur est un espace.
- Utile pour formater la sortie lorsque plusieurs éléments sont imprimés en même temps.
- Peut être défini sur n'importe quelle chaîne de caractères.

```
1 print("Hello", "world", sep=", ")
```



The screenshot shows a Windows command prompt window. At the top, there are four tabs: PROBLEMS, OUTPUT, PORTS, and SERIAL MONITOR. The OUTPUT tab is selected. The prompt shows the command `1 print("Hello", "world", sep=", ")` being executed. The output of the command is `Hello, world`, demonstrating the effect of the `sep` parameter.

Astuces pour l'interaction avec la console

end :

- Permet de spécifier la chaîne à ajouter à la fin de l'impression.
- Par défaut, la fin de l'impression est une nouvelle ligne ("
").
- Utile pour contrôler le comportement de retour à la ligne après l'impression.
- Peut être défini sur n'importe quelle chaîne de caractères pour personnaliser la fin de l'impression.

```
1 print("Hello", "world", sep=", ", end="!")
```

PROBLEMS OUTPUT PORTS SERIAL MONITOR SQL CONSOLE

Microsoft Windows [version 10.0.19045.4046]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\Alexandre\Desktop\demo_base_python>C:/Use
xe c:/Users/Alexandre/Desktop/demo_base_python/opé
Hello, world!

Exercices

1. Demandez à l'utilisateur son prénom et son nom, puis affichez-les avec un séparateur personnalisé.
2. Affichez un message sur plusieurs lignes avec une fin de ligne personnalisée pour chaque ligne.
3. Demandez à l'utilisateur deux mots et affichez-les sur la même ligne, sans espace entre eux.
4. Demandez à l'utilisateur deux nombres et affichez leur somme avec une fin de ligne personnalisée.
5. Affichez trois valeurs avec un séparateur personnalisé et une fin de ligne personnalisée.

4. Opérateurs

- Introductions aux opérateurs
- Opérateurs arithmétiques
- Raccourcis
- Opérateurs de comparaison
- Opérateurs logiques

Introduction aux opérateurs

Les opérateurs sont des symboles spéciaux qui effectuent des opérations sur des valeurs ou des variables en Python.

Les opérateurs sont utilisés dans les expressions pour effectuer des calculs, des comparaisons et des opérations logiques.

Il existe plusieurs types d'opérateurs en Python, notamment les opérateurs **arithmétiques**, de **comparaison**, **logiques**, **d'appartenance**, et **d'identité**.

Opérateurs arithmétiques

Les opérateurs arithmétiques sont utilisés pour effectuer des opérations mathématiques sur des valeurs numériques.

Opérateur	Description	Exemple
+	Addition	$3 + 5$ donne 8
-	Soustraction	$7 - 2$ donne 5
*	Multiplication	$4 * 6$ donne 24
/	Division	$10 / 2$ donne 5.0 (division flottante)
//	Division entière	$10 // 3$ donne 3 (résultat arrondi à l'entier inférieur)
%	Modulo	$10 \% 3$ donne 1 (reste de la division entière de 10 par 3)
**	Exponentiation	$2 ** 3$ donne 8 (2 élevé à la puissance 3)

Raccourcis

Il est possible de simplifier l'écriture des opérations arithmétiques sur les variables en utilisant des raccourcis syntaxiques. Plutôt que d'écrire des expressions complètes telles que "**variable = variable + valeur**" pour effectuer une opération comme l'addition, la soustraction, la multiplication ou la division sur une variable.

Opération	Avant (Forme étendue)	Après (Forme raccourcie)
Addition	$a = a + 3$	$a += 3$
Soustraction	$a = a - 3$	$a -= 3$
Multiplication	$a = a * 3$	$a *= 3$
Division	$a = a / 3$	$a /= 3$
Division entière	$a = a // 3$	$a //= 3$
Modulo	$a = a \% 3$	$a \% = 3$
Exponentiation	$a = a ** 3$	$a ** = 3$

Opérateurs de comparaison

Les opérateurs de comparaison sont utilisés pour comparer des valeurs et retourner des valeurs booléennes (**True** ou **False**) en fonction du résultat de la comparaison.

Opérateur	Description	Exemple
==	Teste l'égalité	5 == 5 donne True
!=	Teste la différence	5 != 3 donne True
>	Teste si le premier nombre est supérieur	7 > 3 donne True
<	Teste si le premier nombre est inférieur	2 < 4 donne True
>=	Teste si le premier nombre est supérieur ou égal	6 >= 6 donne True
<=	Teste si le premier nombre est inférieur ou égal	3 <= 5 donne True

Opérateurs de comparaison : in

L'opérateur `in` est utilisé pour vérifier si un élément fait partie d'une collection, telle qu'une liste, un tuple, un ensemble ou une chaîne de caractères. Il renvoie également `True` si l'élément est présent dans la collection, et `False` sinon. Pour la démonstration nous utilisons une condition que sera vue au chapitre suivant.

```
1  ma_liste = [1, 2, 3, 4, 5]
2
3  ✓ if 3 in ma_liste:
4      print("L'élément 3 est présent dans la liste.")
5  ✓ else:
6      print("L'élément 3 n'est pas présent dans la liste.")
```

Opérateurs de comparaison : is

L'opérateur `is` est utilisé pour vérifier si deux variables font référence au même emplacement mémoire pour les chaînes de caractères.

Contrairement à l'opérateur `==`, qui vérifie si les chaînes ont la même valeur, l'opérateur `is` vérifie si les deux variables font référence au même objet en mémoire, c'est-à-dire si elles sont identiques en termes d'identité.

Considérons les deux variables de chaîne de caractères suivants :

```
1  chaine_1 = "hello"  
2  chaine_2 = "hello"
```

Opérateurs de comparaison : is

Dans cet exemple, `chaine_1` et `chaine_2` contiennent la même séquence de caractères "Hello", et Python optimise la mémoire pour qu'elles référencent le même objet. Par conséquent, l'expression `chaine_1 is chaine_2` renvoie `True`.

```
1  chaine_1 = "hello"
2  chaine_2 = "hello"
```

En revanche, `chaine_a` et `chaine_b` contiennent des séquences de caractères différentes ("Bonjour" et "Bonjour "), donc elles ne font pas référence au même objet en mémoire. Ainsi, l'expression `chaine_a is chaine_b` renvoie `False`.

```
11 chaine_a = "Bonjour"
12 chaine_b = "Bonjour "
```

Opérateurs de comparaison : is

Dans cet exemple :

```
1  # Cas où l'expression renvoie True
2  chaine_1 = "Hello"
3  chaine_2 = "Hello"
4
5  if chaine_1 is chaine_2:
6      print("Les variables chaine_1 et chaine_2 font référence au même objet en mémoire.")
7  else:
8      print("Les variables chaine_1 et chaine_2 ne font pas référence au même objet en mémoire.")
9
10 # Cas où l'expression renvoie False
11 chaine_a = "Bonjour"
12 chaine_b = "Bonjour "
13
14 if chaine_a is chaine_b:
15     print("Les variables chaine_a et chaine_b font référence au même objet en mémoire.")
16 else:
17     print("Les variables chaine_a et chaine_b ne font pas référence au même objet en mémoire.")
```

Opérateurs logiques

Les opérateurs logiques sont des outils fondamentaux en programmation pour évaluer des conditions complexes.

Les opérateurs logiques sont utilisés pour combiner des expressions booléennes et effectuer des tests de conditions.

Les trois opérateurs logiques de base sont "et" (**and**), "ou" (**or**) et "non" (**not**).

Opérateurs logiques (et / and)

L'opérateur "et" (**and**) permet de vérifier si toutes les expressions booléennes liées sont vraies. Il retourne **True** uniquement si toutes les expressions sont évaluées à **True**, sinon il renvoie **False**.

ET

$A \wedge B$	Vrai	Faux
Vrai	V	F
Faux	F	F

Opérateurs logiques (ou / or)

L'opérateur "ou" (**or**) permet de vérifier si au moins l'une des expressions booléennes liées est vraie. Il retourne **True** si au moins une des expressions est évaluée à **True**, sinon il renvoie **False**.

OU

A v B	Vrai	Faux
Vrai	V	V
Faux	V	F

Opérateurs logiques (non / not)

L'opérateur "non" (not) permet d'inverser le résultat d'une expression booléenne. Par exemple, "not True" renverra False et "not False" renverra True.

NON

	$\neg A$
Vrai	F
Faux	V

Exercices

1. Devinez les résultats des calculs sur les valeurs des variables A, B, C, D, E, F et G à partir des expressions suivantes :

1. $A = 12$

2. $B = 3 * A + 8$

3. $C = B - 2 * A$

4. $D = (C + 25) * B$

5. $E = (A + B) \% 10$

6. $F = (C * D) // (B + 6)$

7. $G = (D + E) // (F - 9)$

2. Utilisez Python pour vérifier vos réponses et afficher les valeurs.

Exercices

3. Considérons : $A = 13$ / $B = 5$ / $C = \text{True}$ / $D = \text{not}(C)$. Donnez le résultat pour chacune de ces instructions :

- $A > 10$) and $(B < 20)$
- $(C \neq 40)$ or $(D \geq 100)$
- $(A == 2)$ and $(\text{not } (B > 15))$
- $(C > 50)$ or $(\text{not } (D < 200))$
- $((A < B)$ and $(C > 30))$ or $(D == 270)$
- $\text{not } ((A * B) > 100)$
- $((B == 15)$ or $((C > 60)$ and $(A < 5)))$
- $((((B == 15)$ or $((C > 60)$ and $(A < 5)))$ and $(A < B))$ or $(\text{not } (B == 9))$



Exercices

4. Imaginez et réalisez une méthode manuelle permettant d'inverser le contenu d'une variable entière sans utiliser de variable temporaire.
5. Écrivez un programme Python convertissant un nombre de secondes en jours, heures, minutes et seconds correspondants. Exemple : 4561 secondes correspondent à 0 jour, 1 heure, 16 minutes et 1 seconde. Réfléchissez à la méthode à utiliser, puis réalisez l'algorithme en Python. Testez-le ensuite pour vérifier sa validité.

5. Structures conditionnelles

- Introductions aux conditions
- Structure if
- Structure if-else
- Structure if-elif-else
- Utilisation de conditions multiples
- L'instruction pass
- Structure match
- Opérateur ternaire
- Structures conditionnelles imbriquées

Introductions aux conditions

Les conditions en Python permettent de contrôler le flux d'exécution d'un programme en évaluant des expressions booléennes. Elles offrent la possibilité d'exécuter des blocs de code spécifiques en fonction de la véracité ou de la fausseté de ces expressions.

Grâce aux structures conditionnelles telles que if, else, elif et match, il est possible de réaliser des actions conditionnelles, de gérer plusieurs cas de figure, et de prendre des décisions basées sur des critères spécifiques.

En outre, les conditions permettent d'introduire de la logique et de la flexibilité dans les programmes, en adaptant le comportement du code en fonction des différentes situations rencontrées lors de l'exécution.

```
1  # Demander à l'utilisateur de saisir un nombre
2  nombre = int(input("Entrez un nombre : "))
3
4  # Vérifier si le nombre est positif, négatif ou nul
5  if nombre > 0:
6      print("Le nombre est positif.")
7  elif nombre < 0:
8      print("Le nombre est négatif.")
9  else:
10     print("Le nombre est nul.")
```

If

La structure **if** est utilisée pour exécuter un bloc de code si une condition est vraie.

Elle permet de réaliser des actions conditionnelles, où le code à l'intérieur du bloc **if** est exécuté uniquement si la condition spécifiée évalue à **True**.

Par exemple :

```
1  # Définir une variable x
2  x = 7
3
4  # Vérifier si la condition x > 5 est vraie
5  if x > 5:
6      print("La condition x > 5 est vraie.")
7      # Bloc de code à exécuter si la condition est vraie
8      # Dans cet exemple, nous imprimons simplement un message
```

If-else

Par exemple :

```
1  # Définir une variable x
2  x = 3
3
4  # Vérifier si la condition x > 5 est vraie
5  ✓ if x > 5:
6      # Bloc de code à exécuter si la condition est vraie
7      print("La condition x > 5 est vraie.")
8  ✓ else:
9      # Bloc de code à exécuter si la condition est fausse
10     print("La condition x > 5 est fausse.")
```

La structure **if-else** permet d'exécuter un bloc de code si une condition est vraie et un autre bloc de code si la condition est fausse.

Cela permet de gérer les deux cas possibles d'une condition de manière efficace.

⚠ Le bloc else est facultatif. Cependant, veiller à le renseigner pour couvrir les différents cas de figures.

If-elif-else

La structure **elif** est utilisée pour évaluer plusieurs conditions de manière séquentielle.

Elle permet de tester une série de conditions dans l'ordre et d'exécuter le bloc de code associé à la première condition vraie. Cela évite d'avoir à utiliser plusieurs instructions if imbriquées.

Par exemple :

```
1  # Définir une variable x
2  x = 7
3
4  # Vérifier si la condition x > 10 est vraie
5  if x > 10:
6      # Si la condition x > 10 est vraie
7      print("La condition x > 10 est vraie.")
8  elif x > 5:
9      # Si la condition x > 10 est fausse mais la condition x > 5 est vraie
10     print("La condition x > 5 est vraie mais x <= 10.")
11 else:
12     # Si toutes les conditions précédentes sont fausses
13     print("Aucune des conditions x > 10 ou x > 5 n'est vraie.")
```

Utilisation de conditions multiples

Python permet d'utiliser des conditions multiples dans une seule instruction if en utilisant les opérateurs logiques **and**, **or** et **not**. Cela permet de combiner plusieurs conditions pour obtenir des résultats plus complexes.

Par exemple :

```
1  # Définir les valeurs de x et y
2  x = 5
3  y = 7
4
5  # Vérifier si à la fois x est supérieur à 0 et y est inférieur à 10
6  if x > 0 and y < 10:
7      # Bloc de code à exécuter si les deux conditions sont vraies
8      print("Les deux conditions sont vraies : x > 0 et y < 10.")
9  else:
10     # Bloc de code à exécuter si au moins l'une des conditions est fausse
11     print("Au moins l'une des conditions n'est pas remplie.")
```

Pass

`pass` est utilisée comme un espace réservé dans le code. Elle ne fait rien et sert simplement à indiquer qu'aucune action n'est nécessaire à cet endroit.

L'utilisation principale de `pass` est de permettre la définition de structures de contrôle, de fonctions ou de classes vides sans générer d'erreur de syntaxe.

Cela peut être utile lors de l'écriture de code qui doit être rempli ultérieurement ou lors de la définition de blocs de code conditionnels ou de boucles qui ne nécessitent pas d'action immédiate.

```
1  # Définir une variable booléenne
2  condition = False
3
4  # Vérifier si la condition est vraie
5  if condition:
6      pass # Aucune action à effectuer si la condition est vraie
7  else:
8      print("La condition est fausse.")
```

Structure match

l'instruction **match** est utilisée pour effectuer des comparaisons de motifs (pattern matching).

Introduite dans Python 3.10, cette fonctionnalité offre une manière plus concise et lisible de comparer une valeur à plusieurs alternatives, en remplaçant la syntaxe parfois complexe des instructions **if**, **elif**, **else**.

Le **match** s'apparente à une série d'instructions **if-elif-else**, mais avec une syntaxe plus expressive et une vérification exhaustive des alternatives.

```
1  valeur = 5
2
3  # Utilisation de l'instruction match pour vérifier la valeur de la variable 'valeur'
4  match valeur:
5      case 1:
6          # Si 'valeur' est égale à 1, affiche ce message
7          print("La valeur est égale à 1")..
8      case 2:
9          # Si 'valeur' est égale à 2, affiche ce message
10         print("La valeur est égale à 2")..
11     case 3:
12         # Si 'valeur' est égale à 3, affiche ce message
13         print("La valeur est égale à 3")..
14     case _: # Le "_" capture toutes les autres valeurs
15         # Si 'valeur' est différente de 1, 2, ou 3, affiche ce message
16         print("La valeur n'est ni 1, ni 2, ni 3")..
```

Opérateur ternaire

L'opérateur **ternaire** est une expression conditionnelle compacte qui permet d'écrire du code concis pour évaluer une condition.

Il est souvent utilisé lorsque vous avez une condition simple avec deux résultats possibles.

Par exemple:

```
1  # Définir une variable x
2  x = 7
3
4  # Utiliser l'opérateur ternaire pour assigner une valeur
5  # à une variable en fonction d'une condition
6  resultat = "x est supérieur à 5" if x > 5 else "x est inférieur ou égal à 5"
7
8  # Afficher le résultat
9  print(resultat)
```

Structure conditionnelles imbriquées

Les structures conditionnelles imbriquées sont des structures `if`, `elif` et `else` qui sont imbriquées à l'intérieur d'autres structures conditionnelles.

Cela permet de traiter des conditions multiples et des cas spécifiques de manière hiérarchique.

Il est important de maintenir la lisibilité du code lors de l'utilisation de structures conditionnelles imbriquées pour éviter la complexité excessive.

```
1  # Définir une variable x
2  x = 10
3
4  # Vérifier la valeur de x
5  if x > 0:
6      print("x est positif.")
7      if x % 2 == 0:
8          print("x est un nombre pair.")
9      else:
10         print("x est un nombre impair.")
11 else:
12     print("x est négatif ou nul.")
```

Exercices

1. Créez un programme qui gère les commandes de café en fonction des différentes options telles que la taille, le type de café, les extras, etc. Utilisez des correspondances pour traiter chaque option et calculer le prix total de la commande.
2. Créez un programme qui convertit une note numérique en une note alphabétique en utilisant une échelle de notation standard. Utilisez des correspondances pour déterminer la note alphabétique correspondante en fonction de la note numérique.
3. Créez un programme qui génère un nombre aléatoire (`import random`) et permet à l'utilisateur de deviner ce nombre. Utilisez des correspondances pour comparer la devinette de l'utilisateur avec le nombre généré et fournir des indices.

Exercices

4. Créez un programme qui calcule l'indice de masse corporelle (IMC) d'une personne en fonction de son poids et de sa taille. Utilisez des correspondances pour interpréter et catégoriser l'IMC résultant en différentes catégories de poids.
5. Créez un programme qui permet à l'utilisateur de choisir un menu pour chaque repas (petit-déjeuner, déjeuner, dîner) parmi des options préétablies. Après la sélection, il affiche les choix de l'utilisateur pour chaque repas et résume l'ensemble des repas de la journée.
6. Créez un programme qui génère des citations aléatoires à partir d'un nombre aléatoire. L'utilisateur devrait pouvoir choisir un thème et le programme générera une citation aléatoire correspondante. Utilisez des correspondances pour gérer les différents thèmes et générer les citations appropriées.

6. Structures itératives

- Introduction aux boucles
- Précautions
- While
- For
- Cas d'utilisation
- Break & continue
- Bonnes pratiques

Introduction aux boucles

Les structures itératives sont des éléments essentiels de la programmation, offrant la capacité de répéter des instructions de manière efficace et contrôlée.

En Python, les boucles `while` et `for` sont les deux principales structures itératives utilisées pour parcourir des données, automatiser des tâches et mettre en place la répétitions.

Il est important de garder également à l'esprit quelques points essentiels lors de l'utilisation des structures itératives comme nous allons le voir dans le slide suivant.

Précautions

- **Condition de sortie** : Lors de l'utilisation d'une boucle **while**, il est crucial de s'assurer qu'il existe une condition de sortie qui finira par devenir fausse. Sinon, la boucle pourrait s'exécuter indéfiniment, entraînant ce qu'on appelle une boucle infinie.
- **Complexité algorithmique** : Les boucles peuvent avoir un impact significatif sur la performance de votre programme, surtout si elles sont utilisées de manière inefficace. Essayez d'éviter les boucles imbriquées excessivement profondes ou les boucles avec un grand nombre d'itérations, car cela peut ralentir considérablement l'exécution du programme.
- **Clarté du code** : Lorsque vous utilisez des boucles, assurez-vous que votre code est clair et facile à comprendre. Évitez les boucles trop complexes et essayez de décomposer les tâches en fonctions ou en blocs de code plus petits si nécessaire.

While

La boucle **while** est une structure itérative utilisée pour exécuter un bloc de code tant qu'une condition spécifique reste vraie.

- Tout d'abord, Python évalue la condition de la boucle **while**.
- Si cette condition est vraie, le bloc de code à l'intérieur est exécuté.
- Après l'exécution du bloc de code, la condition est à nouveau évaluée.
- Si la condition reste vraie, le bloc de code est à nouveau exécuté.
- Ce processus se répète jusqu'à ce que la condition devienne fausse. .

```
1 i = 0
2 while i < 5:
3     print("Je suis dans la boucle")
4     i += 1
5
6 print("Je suis sorti de la boucle")
```

Une fois que la condition devient fausse, l'exécution de la boucle **while** se termine et le programme continue son exécution normale à partir de l'instruction suivant la boucle

For

Le boucle **for** en Python offre une puissante capacité à itérer sur des séquences de données telles que des listes, des tuples, des ensembles ou des chaînes de caractères.

En utilisant une boucle **for**, il est possible d'accéder à chaque élément de la séquence un par un, ce qui facilite le traitement des données de manière séquentielle.

Cette structure itérative permet d'automatiser des tâches répétitives, telles que le calcul de la somme des éléments d'une liste, la recherche d'un élément spécifique dans une séquence ou la manipulation de chaînes de caractères.

De plus, le boucle **for** peut être combinée avec des instructions conditionnelles et d'autres fonctionnalités de Python pour créer des algorithmes complexes et des programmes puissants.

Cas d'utilisation : collection

L'utilisation d'une boucle **for** à travers une collection est une méthode courante pour parcourir chaque élément de la collection et effectuer des opérations sur ces éléments.

À chaque itération de la boucle **for**, la variable **nombre** prend la valeur de l'élément actuellement parcouru dans la collection **liste_nombres**. La boucle continue à itérer sur chaque élément de la collection jusqu'à ce qu'elle atteigne la fin de celle-ci.

```
1  # Définition d'une liste de nombres
2  liste_nombres = [1, 2, 3, 4, 5]
3
4  # Utilisation d'une boucle for pour parcourir chaque élément de la liste
5  print("Parcours de la liste de nombres :")
6  for nombre in liste_nombres:
7      print(nombre)
```

Cas d'utilisation : enumerate()

La méthode `enumerate()` est une fonction intégrée qui permet d'itérer simultanément sur les éléments et leurs indices dans une collection. Dans cette syntaxe :

- `index` est une variable qui prend la valeur de l'indice de chaque élément dans la collection.
- `element` est une variable qui prend la valeur de chaque élément de la collection à chaque itération de la boucle.
- `ma_liste` est la collection de données à parcourir, telle qu'une liste, un tuple, un ensemble ou une chaîne de caractères.

Les instructions indentées sous la boucle `for` sont exécutées pour chaque élément de la collection, avec accès à la fois à l'élément lui-même et à son indice.

```
1  # Définition d'une liste d'éléments
2  ma_liste = ['a', 'b', 'c', 'd', 'e']
3
4  print("Les éléments de la liste avec leurs indices sont :")
5  for index, element in enumerate(ma_liste):
6      print(f"L'élément à l'indice {index} est : {element}")
```

```
exe c:/Users/Alexandre/Desktop/demo_base_python/demo
Les éléments de la liste avec leurs indices sont :
L'élément à l'indice 0 est : a
L'élément à l'indice 1 est : b
L'élément à l'indice 2 est : c
L'élément à l'indice 3 est : d
L'élément à l'indice 4 est : e
```

Cas d'utilisation : range()

La méthode `range()` est une fonction intégrée également qui génère une séquence d'entiers dans un intervalle spécifié. Cette fonction est couramment utilisée avec la boucle `for` pour itérer un nombre précis de fois ou pour générer des séquences d'indices lors du parcours de collections de données. Dans cette syntaxe :

- `start` est l'entier de départ de la séquence (par défaut, il est égal à 0).
- `stop` est l'entier de fin de la séquence (non inclus).
- `step` est le pas (incrément) entre chaque entier de la séquence (par défaut, il est égal à 1).

```
1  start = 0
2  stop = 3
3  step = 1
4
5  for i in range(start, stop, step):
6      print(i)
```

```
exe c:/Users/Alexandre/Desktop/demo
0
1
2
```

Continue

L'instruction **continue** est utilisée à l'intérieur d'une boucle pour passer à l'itération suivante sans exécuter le reste du code dans le bloc de la boucle actuelle.

Cela permet d'ignorer certaines itérations de la boucle en fonction d'une condition spécifique, sans arrêter complètement l'exécution de la boucle.

Par exemple, dans une boucle **for** ou **while**, si une condition est remplie, continue permet de passer à l'itération suivante, en sautant le reste du bloc de code de la boucle pour cette itération.

```
1  for i in range(10):  
2      if i % 2 == 0:  
3          continue # Passe à l'itération suivante si i est pair  
4      print(i) # Affiche uniquement les nombres impairs
```

Break

L'instruction **break** en Python est un outil puissant pour contrôler le flux d'exécution des boucles. Lorsqu'elle est rencontrée à l'intérieur d'une boucle, telle qu'une boucle **for** ou **while**, elle provoque immédiatement la sortie anticipée de la boucle, interrompant ainsi son exécution normale.

Cette fonctionnalité est particulièrement utile lorsque nous devons arrêter une boucle prématurément dès qu'une condition spécifique est satisfaite. Cela permet d'éviter l'exécution de code superflu et d'améliorer l'efficacité de notre programme.

```
1  # Définition d'une liste de nombres
2  liste_nombres = [1, 2, 3, 4, 5]
3
4  # Parcours de la liste de nombres
5  for nombre in liste_nombres:
6      print(nombre) # Affichage du nombre
7
8      # Si le nombre est égal à 3, sortir de la boucle
9      if nombre == 3:
10         print("Condition remplie. Sortie de la boucle.")
11         break
```

Bonnes pratiques

Break :

- Utilisez **break** avec parcimonie pour éviter les sorties abruptes des boucles, préférant des structures de contrôle plus expressives lorsque cela est possible.
- Assurez-vous que l'utilisation de **break** est claire et justifiée, généralement pour terminer prématurément une boucle lorsqu'une condition particulière est remplie.

Continue :

- Utilisez **continue** avec modération pour éviter des structures de boucle complexes et difficiles à suivre.
- Utilisez **continue** pour sauter des itérations spécifiques de la boucle de manière explicite, en veillant à ce que la logique de votre boucle reste claire et compréhensible.

Exercices

1. Concevez un programme qui génère et affiche les nombres premiers jusqu'à 100 en utilisant une boucle.
2. Créez un programme qui demande à l'utilisateur d'entrer son âge. Utilisez un opérateur ternaire pour vérifier si l'utilisateur est majeur ou mineur. Affichez ensuite un message approprié en fonction de la réponse.
3. **Jeu de devinette de nombre amélioré** : Écrivez un jeu interactif où l'ordinateur génère un nombre aléatoire entre 1 et 100, et l'utilisateur doit deviner ce nombre. Utilisez une boucle pour permettre à l'utilisateur de faire plusieurs tentatives. Après chaque tentative, demandez à l'utilisateur s'il souhaite continuer à jouer. Répétez le processus jusqu'à ce qu'il décide de ne plus jouer. Enfin, affichez le nombre de tentatives utilisées pour deviner le nombre.
4. Écrivez un programme qui demande à l'utilisateur d'entrer un mot. Utilisez une boucle pour afficher chaque caractère du mot un par un jusqu'à la fin du mot.

Exercices

1. **Gestionnaire de commandes de café amélioré :** Écrivez un programme interactif qui prend les commandes de café en fonction des options telles que la taille, le type de café et les extras. Utilisez une boucle pour permettre à l'utilisateur de passer plusieurs commandes et affichez le prix total à la fin de chaque commande. Demandez ensuite à l'utilisateur s'il souhaite passer une autre commande et répétez le processus jusqu'à ce qu'il n'en ait plus envie.
2. Écrivez un programme qui demande à l'utilisateur d'entrer un mot. Utilisez une boucle pour inverser l'ordre des lettres du mot et affichez le mot inversé à la fin.
3. **Gestion des repas du jour améliorée :** Développez un programme interactif qui permet à l'utilisateur de choisir un menu pour chaque repas de la journée parmi des options prédéfinies. Utilisez une boucle pour faciliter la saisie des choix pour chaque repas et affichez un résumé des choix de repas à la fin. Demandez ensuite à l'utilisateur s'il souhaite choisir les repas pour un autre jour. Répétez le processus jusqu'à ce qu'il décide de ne plus choisir de repas.

7. Collections

- Introduction aux collections
- List
- Tuple
- Set
- Dictionnaire
- Résumé
- Trucs et astuces

Introduction aux collections

Les collections en Python sont des structures de données fondamentales qui permettent de stocker, organiser et manipuler des données de manière efficace dans un programme. Elles offrent une variété d'outils pour répondre à divers besoins en matière de traitement de données.

Ces collections sont utilisées dans une multitude de scénarios de programmation, que ce soit pour stocker des données, effectuer des opérations ensemblistes, ou pour structurer des informations de manière claire et accessible.

Liste

Les **listes** permettent de stocker des séquences modifiables d'éléments. Elles peuvent contenir différents types de données et sont définies en encadrant les éléments entre crochets et en les séparant par des virgules.

```
1  ma_liste = list(1, True, list(), tuple(), "ect...")
```

Liste

Lorsque vous déclarez une **liste**, nous utilisons des crochets `[]` pour encadrer les éléments de la liste et les séparez par des virgules. Par exemple :

```
1  ma_liste_explicite = [1, 2, 3, 4, 5]
```

Ici, les crochets `[]` indiquent clairement que vous créez une **liste**, et les virgules séparent les éléments individuels.

Liste

Une autre façon de le déclarer consiste à utiliser le constructeur `list()` en passant éventuellement une séquence comme argument, telle qu'une chaîne de caractères ou un tuple. Par exemple, `ma_liste = list("hello")` crée une liste avec chaque caractère de la chaîne "hello" comme élément.

```
1  # Déclaration d'une chaîne de caractères
2  chaine = "hello"
3
4  # Utilisation de la fonction list()
5  ma_liste = list(chaine)
6
7  # Affichage de la liste résultante
8  print(ma_liste)
```

PROBLEMS OUTPUT PORTS SERIAL MONITOR SQL CONSOLE

```
C:\Users\Alexandre\Desktop\demo_base_python>C:/Users/Alexandre/Desktop/demo_base_python/d
['h', 'e', 'l', 'l', 'o']
```

Liste

Ces listes sont accompagnées de nombreuses méthodes intégrées pour faciliter leur manipulation. Parmi celles-ci, on trouve :

Méthode	Description	Exemple
<code>append()</code>	Ajoute un élément à la fin de la liste	<code>ma_liste.append(10)</code>
<code>insert()</code>	Insère un élément à une position spécifiée	<code>ma_liste.insert(2, 'élément')</code>
<code>extend()</code>	Étend la liste en ajoutant les éléments d'une autre liste	<code>ma_liste.extend([6, 7, 8])</code>
<code>remove()</code>	Supprime la première occurrence d'un élément	<code>ma_liste.remove('élément')</code>
<code>pop()</code>	Supprime et renvoie l'élément à une position spécifiée	<code>ma_liste.pop(1)</code>
<code>index()</code>	Retourne l'indice de la première occurrence d'un élément	<code>indice = ma_liste.index('élément')</code>

Liste

count()	Compte le nombre d'occurrences d'un élément dans la liste	<code>nombre = ma_liste.count('élément')</code>
sort()	Trie les éléments de la liste par ordre croissant (ou personnalisé)	<code>ma_liste.sort()</code>
reverse()	Inverse l'ordre des éléments de la liste	<code>ma_liste.reverse()</code>
clear()	Supprime tous les éléments de la liste	<code>ma_liste.clear()</code>
copy()	Renvoie une copie superficielle de la liste	<code>nouvelle_liste = ma_liste.copy()</code>
len()	Retourne le nombre d'éléments dans la liste	<code>longueur = len(ma_liste)</code>

Tuple

Les **tuples** sont des structures de données immuables, ce qui signifie qu'une fois créés, ils ne peuvent pas être modifiés. Les **tuples** sont généralement utilisés pour stocker des collections d'éléments hétérogènes (c'est-à-dire contenant différents types de données) dans un ordre spécifique.

```
1 mon_tuple = (1, True, [], (), "ect...")
```

Tuple

Python peut comprendre dans les cas d'un **tuple** la déclaration explicite ainsi que implicite. Cependant le symbole utilisé pour la déclaration de celle-ci sera différent. En effet nous utiliserons des `()` a la place des `[]`

```
1 mon_tuple_implicite = 1, 2, 3
2
3 mon_tuple_explicite = (1, 2, 3)
```

Set

Les ensembles, représentés par le type de données **set**, offrent une structure de données permettant de stocker des éléments uniques dans un ordre non défini. Ils sont particulièrement utiles lorsque vous avez besoin de collecter des éléments uniques ou d'éliminer les doublons d'une collection de données.

Les ensembles permettent des opérations ensemblistes telles que l'union, l'intersection, la différence et la différence symétrique, ce qui facilite la comparaison et la manipulation de données entre plusieurs ensembles.

En outre, les ensembles sont mutables, ce qui signifie que vous pouvez ajouter et supprimer des éléments à tout moment.

Set

Il existe plusieurs façons de déclarer un ensemble :

- **Déclaration explicite avec des accolades** : déclarer un ensemble de manière explicite en utilisant des accolades `{}` et en séparant les éléments par des virgules
- **Utilisation du constructeur `set()` avec une liste** : créer un ensemble à partir d'une liste en utilisant le constructeur `set()`.
- **Utilisation du constructeur `set()` avec un tuple** : Vous pouvez créer un ensemble en utilisant le constructeur `set()` avec un **tuple** contenant des éléments individuels.

```
1  ensemble = {1, 2, 3, 4}
```

```
2  ensemble = set([1, 2, 3, 4])
```

```
3  ensemble = set((1, 2, 3, 4))
```

Set

Tout comme les **listes** sont dotées de nombreuses méthodes intégrées facilitant leur manipulation, les ensembles (**sets**) en Python bénéficient également d'un ensemble de méthodes intégrées.

Méthode	Description	Exemple
<code>add()</code>	Ajoute un élément à l'ensemble.	<code>my_set.add()</code>
<code>remove()</code>	Supprime un élément de l'ensemble.	<code>my_set.remove()</code>
<code>discard()</code>	Supprime un élément de l'ensemble s'il existe.	<code>my_set.discard()</code>
<code>pop()</code>	Supprime et retourne un élément de l'ensemble (aléatoirement).	<code>my_set.pop()</code>
<code>clear()</code>	Supprime tous les éléments de l'ensemble.	<code>my_set.clear()</code>
<code>copy()</code>	Retourne une copie de l'ensemble.	<code>my_set.copy()</code>

Set

union()	Retourne l'union de deux ensembles.	<code>set1.union(set2)</code>
intersection()	Retourne l'intersection de deux ensembles.	<code>set1.intersection(set2)</code>
difference()	Retourne la différence entre deux ensembles.	<code>set1.difference(set2)</code>
symmetric_difference()	Retourne la différence symétrique entre deux ensembles.	<code>set1.symmetric_difference(set2)</code>
isdisjoint()	Vérifie si deux ensembles n'ont aucun élément en commun.	<code>set1.isdisjoint(set2)</code>
issubset()	Vérifie si un ensemble est un sous-ensemble d'un autre.	<code>set1.issubset(set2)</code>
issuperset()	Vérifie si un ensemble contient tous les éléments d'un autre ensemble.	<code>set1.issuperset(set2)</code>

Dictionnaires

Les **dictionnaires** sont des structures de données flexibles qui associent des clés uniques à des valeurs correspondantes, permettant un stockage et un accès efficaces aux informations.

Leur mutabilité autorise l'ajout, la suppression et la modification d'éléments après leur création, offrant ainsi une grande adaptabilité. Ces structures peuvent contenir un nombre variable d'éléments et sont extensibles, permettant l'ajout de nouvelles paires clé-valeur au besoin.

Grâce à leur flexibilité en termes de types de données, les **dictionnaires** sont capables de stocker une variété d'informations diverses de manière organisée.

Dictionnaires

Il existe plusieurs façons de déclarer un dictionnaire :

- Déclaration explicite avec des accolades `{}` : déclarer un dictionnaire en énumérant les paires clé-valeur entre accolades `{}` et en les séparant par des virgules. Par exemple :
- Utilisation du constructeur `dict()` avec des paires clé-valeur : créer un **dictionnaire** en utilisant le constructeur `dict()` et en passant potentiellement des paires clé-valeur sous forme de tuples.

```
1 dictionnaire = {  
2     "clé1": "valeur1",  
3     "clé2": "valeur2"  
4 }
```

```
7 dictionnaire = dict([  
8     ('clé1', "valeur1"),  
9     ('clé2', "valeur2")  
10 ])
```

Dictionnaires

Utilisation de la syntaxe de compréhension de **dictionnaire** :
créer un dictionnaire en utilisant la syntaxe de
compréhension de dictionnaire, qui permet de générer un
dictionnaire à partir d'une itération sur une séquence.

```
1  dictionnaire = {i: i*2 for i in range(5)}
2
3  print(dictionnaire)
4
```

PROBLEMS TERMINAL ... Python + - [] [X] ...

Microsoft Windows [version 10.0.19045.4046]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\Caven\Desktop\demo>C:/Users/Caven/AppData/Local/Programs/Python/Python312/python.exe c:/Users/Caven/Desktop/demo/dict.py

{0: 0, 1: 2, 2: 4, 3: 6, 4: 8}

Dictionnaires

les **dictionnaires** en Python disposent également d'un ensemble de méthodes intégrées qui offrent une grande souplesse dans leur utilisation et leur manipulation.

Méthode	Description	Exemple
<code>clear()</code>	Supprime tous les éléments du dictionnaire	<code>mon_dictionnaire.clear()</code>
<code>copy()</code>	Renvoie une copie superficielle du dictionnaire	<code>mon_dictionnaire.copy()</code>
<code>get()</code>	Renvoie la valeur associée à une clé spécifiée	<code>mon_dictionnaire.get('clé')</code>
<code>items()</code>	Renvoie une vue sur les paires (clé, valeur) du dictionnaire	<code>mon_dictionnaire.items()</code>
<code>keys()</code>	Renvoie une vue sur les clés du dictionnaire	<code>mon_dictionnaire.keys()</code>

Dictionnaires

pop()	Supprime et renvoie la valeur associée à une clé spécifiée	<code>mon_dictionnaire.pop('clé')</code>
popitem()	Supprime et renvoie une paire (clé, valeur) au hasard	<code>mon_dictionnaire.popitem()</code>
setdefault()	Renvoie la valeur associée à une clé, en l'ajoutant si elle n'existe pas	<code>mon_dictionnaire.setdefault('clé', 'valeur_par_défaut')</code>
update()	Met à jour le dictionnaire avec les paires (clé, valeur) d'un autre dictionnaire ou d'une séquence de paires	<code>mon_dictionnaire.update({'clé': 'nouvelle_valeur'})</code>
values()	Renvoie une vue sur les valeurs du dictionnaire	<code>valeurs = mon_dictionnaire.values()</code>

Résumé

Collection	Ordre	Duplication	Modifiable	Itérable	Indexable	Clé-Valeur	Immutable
Liste	Oui	Oui	Oui	Oui	Oui	Non	Non
Tuple	Oui	Oui	Non	Oui	Oui	Non	Oui
Set	Non	Non	Oui	Oui	Non	Non	Non
Dictionnaire	Non	Non	Oui	Oui	Non	Oui	Non

Trucs et astuces

Pour récupérer un intervalle de valeurs à partir d'une liste en Python, nous pouvons utiliser la syntaxe de découpage (**slicing**). Cela nous permet de spécifier un début, une fin et un pas pour extraire une partie spécifique de la liste.

Syntaxe	Description	Exemple
<code>ma_liste[start:stop]</code>	Sélectionne les éléments de start à stop-1 (exclus).	<code>ma_liste[2:5]</code>
<code>ma_liste[start:stop:step]</code>	Sélectionne les éléments de start à stop-1 avec un pas de step.	<code>ma_liste[1:8:2]</code>
<code>ma_liste[start:]</code>	Sélectionne les éléments de start jusqu'à la fin de la liste.	<code>ma_liste[2:]</code>
<code>ma_liste[:stop]</code>	Sélectionne les éléments du début jusqu'à stop-1 (exclus).	<code>ma_liste[:5]</code>
<code>ma_liste[::step]</code>	Sélectionne tous les éléments de la liste avec un pas de step.	<code>ma_liste[::2]</code>
<code>ma_liste[:]</code>	Sélectionne tous les éléments de la liste.	<code>ma_liste[:]</code>

Trucs et astuces

Python offre une syntaxe élégante pour créer des listes avec des opérations, des boucles et des conditions connues sous le nom de compréhensions de liste. Ces constructions permettent de simplifier le processus de création de listes en exprimant les opérations de manière concise et claire.

```
1  # Création d'une liste avec une opération
2  ma_liste_de_nombres = [1, 2, 3, 4, 5]
3  # Liste des nombres, augmentés de 3
4  liste_3 = [n + 3 for n in ma_liste_de_nombres]
5  print(liste_3)  # Résultat : [4, 5, 6, 7, 8]
6
7  # Somme des carrés des nombres de 1 à 10
8  entier_1 = sum([n**2 for n in range(1, 11)])
9  print(entier_1)  # Résultat : 385
```

```
1  # Compréhension avec condition
2  # Liste contenant les carrés des nombres naturels pairs jusqu'à 16 exclu
3  liste_4 = [n**2 for n in range(16) if n % 2 == 0]
4  print(liste_4)  # Résultat : [0, 4, 16, 36, 64, 100, 144, 196, 256]
```

Exercices

1. Commencez par créer une liste de 10 nombres aléatoires compris entre 1 et 100. Ensuite, affichez cette liste générée. Après cela, calculez la somme de tous les éléments de la liste et affichez-la.
2. Invitez l'utilisateur à saisir son prénom et son nom. Ensuite, créez un tuple contenant ces informations. Pour finir, affichez séparément le prénom et le nom à partir du tuple.
3. Générez deux ensembles de nombres aléatoires compris entre 1 et 20. Affichez ces deux ensembles générés. Enfin, trouvez l'intersection des deux ensembles et affichez-la.
4. Créez un dictionnaire contenant les prix de quelques fruits tels que la pomme, la banane et l'orange. Demandez à l'utilisateur de saisir le nom d'un fruit, puis affichez le prix correspondant à ce fruit s'il existe dans le dictionnaire.

Exercices

5. Créez une liste de tuples contenant le nom et l'âge de trois personnes. Trouvez ensuite la personne la plus âgée et affichez son nom.
6. Générez une liste de 10 nombres aléatoires compris entre 1 et 50. Affichez cette liste générée. Ensuite, filtrez les nombres pairs de la liste et créez une nouvelle liste ne contenant que ces nombres pairs. Enfin, affichez la nouvelle liste contenant uniquement les nombres pairs.
7. Créez une liste de mots contenant des doublons. Transformez ensuite cette liste en un ensemble pour éliminer les doublons. Affichez l'ensemble résultant.

Exercices

8. Créez un dictionnaire de listes représentant différents cours et les étudiants inscrits dans chaque cours. Ajoutez des étudiants à chaque cours. Ensuite, demandez à l'utilisateur de saisir le nom d'un cours et affichez la liste des étudiants inscrits à ce cours.
9. Créez une liste de tuples représentant les commandes d'achat avec les produits et les quantités. Ensuite, créez un dictionnaire de prix pour chaque produit. Calculez ensuite le coût total de toutes les commandes et affichez-le.
10. Créez une liste de dictionnaires représentant les informations des employés avec leur nom, salaire et département. Calculez la somme des salaires pour chaque département, puis calculez la moyenne des salaires pour chaque département. Enfin, affichez les moyennes des salaires pour chaque département.

8. Fonctions & procédures

- Introduction aux fonctions & procédures
- Syntaxe de base
- Arguments optionnels
- Portée des variables
- Fonctions intégrées
- Fonctions lambda
- Documentation de fonctions
- Pratiques de programmation

Introduction aux fonctions & procédures

Les **fonctions** sont des blocs de code nommés et réutilisables qui effectuent une tâche spécifique. Elles permettent d'organiser le code en le divisant en tâches plus petites et plus gérables, favorisant ainsi la modularité et la réutilisabilité.

Elles sont essentielles dans la programmation pour éviter la redondance, améliorer la lisibilité et faciliter la maintenance du code.

En définissant des **fonctions** pour des opérations courantes ou répétitives, nous pourrions encapsuler la logique métier de manière claire et cohérente, ce qui facilite la compréhension et la collaboration dans les projets de développement.

Syntaxe de base

Le mot-clé **def** en Python est utilisé pour définir à la fois des fonctions et des procédures. Il sert à marquer le début de la définition d'une fonction ou d'une procédure.

L'utilisation de **def** permet de spécifier le nom de la **fonction** ou de la **procédure**, ainsi que les paramètres éventuels pour les **fonctions**. Il indique également le bloc de code qui constitue le corps de la **fonction** ou de la **procédure**.

```
1  def fonction():  
2      pass
```

Syntaxe de base

la principale différence entre les **fonctions** et les **procédures** réside dans leur retour de valeur.

Les **fonctions** retournent une valeur après leur exécution, tandis que les **procédures** ne retournent rien.

Par exemple, une **fonction** peut calculer la somme de deux nombres et renvoyer le résultat, tandis qu'une **procédure** peut simplement afficher un message à l'écran sans retourner de valeur.

Fonction :

```
1 def calcul_somme(a, b):  
2     return a + b  
3  
4 resultat = calcul_somme(3, 5)  
5 print("La somme est :", resultat)
```

Procédure :

```
8 def afficher_message():  
9     print("Bonjour, bienvenue !")  
10  
11 afficher_message()
```

Syntaxe de base

les **fonctions** peuvent accepter des paramètres, également appelés arguments, qui sont des valeurs transmises à la fonction lors de son appel.

Les paramètres sont spécifiés entre parenthèses dans la déclaration de la fonction. Par exemple, une **fonction** **calculer_somme** peut prendre deux paramètres, **a** et **b**, qui représentent les nombres à additionner.

Lors de l'appel de cette fonction avec les valeurs spécifiées pour **a** et **b**, ces valeurs sont transmises en tant qu'arguments.

```
1 def calcul_somme(a, b):  
2     return a + b  
3  
4 resultat = calcul_somme(3, 5)  
5 print("La somme est :", resultat)
```

Arguments optionnels

Les paramètres optionnels sont une fonctionnalité puissante en Python, permettant aux **fonctions** d'accepter un nombre variable d'arguments avec des valeurs par défaut.

Cela offre une grande flexibilité lors de l'appel des **fonctions**, car les arguments optionnels peuvent être omis lors de l'appel, prenant ainsi leur valeur par défaut définie dans la déclaration de la fonction.

Par exemple, une **fonction** peut avoir des paramètres optionnels pour personnaliser son comportement, mais ces paramètres ne sont pas obligatoires.

```
1 def bienvenue(nom, message="Hello"):  
2     print(message, nom)  
3  
4 # Appel de la fonction avec un seul argument  
5 bienvenue("Alex") # Affiche: Hello Alex  
6  
7 # Appel de la fonction avec deux arguments  
8 bienvenue("John", "Bonjour") # Affiche: Bonjour John
```

Portée des variables

les variables peuvent être locales à une fonction (variables locales)
ou globales à l'ensemble du programme (variables globales).

Il est important de comprendre la portée des variables pour éviter
les conflits et les erreurs de nommage.

Les variables locales sont définies à l'intérieur de la fonction et ne
sont accessibles que dans celle-ci, tandis que les variables
globales sont définies en dehors de toutes les fonctions et peuvent
être utilisées dans l'ensemble du programme.

```
1  # Variable globale
2  global_var = 10
3
4  def my_function():
5      # Variable locale à la fonction
6      local_var = 20
7      print("Variable locale à la fonction :", local_var)
8      print("Variable globale :", global_var)
9
10 # Appel de la fonction
11 my_function()
12
13 # causera une erreur
14 print("Variable locale en dehors de la fonction :", local_var)
```

Fonctions intégrées

Les fonctions intégrées, également appelées fonctions natives, sont des fonctions pré-définies dans Python qui sont disponibles pour une utilisation immédiate sans avoir besoin d'importer des modules supplémentaires.

Elles font partie du noyau de Python et offrent des fonctionnalités de base pour effectuer diverses opérations, telles que la conversion de types de données, le tri de séquences, le calcul de la longueur d'une séquence, et bien plus encore.

Ces fonctions sont largement utilisées dans le développement Python en raison de leur commodité et de leur efficacité, offrant des outils puissants pour manipuler des données et effectuer des opérations courantes.

Fonctions intégrés

Voici quelques-unes des fonctions intégrées couramment utilisées en Python :

Fonction	Description	Exemple
<code>len()</code>	Retourne la longueur d'une séquence	<code>len([1, 2, 3])</code> donne 3
<code>sorted()</code>	Trie les éléments d'une séquence	<code>sorted([3, 1, 2])</code> donne [1, 2, 3]
<code>min()</code>	Retourne le plus petit élément d'une séquence	<code>min([3, 1, 2])</code> donne 1
<code>max()</code>	Retourne le plus grand élément d'une séquence	<code>max([3, 1, 2])</code> donne 3
<code>map()</code>	Applique une fonction à chaque élément d'une séquence	<code>map(lambda x: x*2, [1, 2, 3])</code> donne [2, 4, 6]
<code>ascii()</code>	Retourne une représentation ASCII d'un objet	<code>ascii('é')</code> donne <code>'\xe9'</code>

<https://docs.python.org/fr/3/library/functions.html>

Fonctions lambda

Les fonctions lambda, également appelées fonctions anonymes, sont des fonctions de courte durée qui peuvent être définies en ligne sans avoir besoin d'être nommées explicitement. Elles sont souvent utilisées pour des opérations simples et sont particulièrement utiles lorsqu'on a besoin de passer une fonction en tant qu'argument à une autre fonction.

```
1  # Addition de deux nombres à l'aide d'une fonction lambda
2  addition = lambda x, y: x + y
3  resultat = addition(3, 5)
4  print(resultat)  # Affichage: 8
```

Documentation de fonctions

Documenter les fonctions en Python est une pratique essentielle pour améliorer la lisibilité et la maintenance du code. Les commentaires, sous forme de docstrings, sont placés juste après la signature de la fonction et fournissent des informations détaillées sur son utilisation, ses paramètres, ses valeurs de retour et d'autres détails importants.

```
1 def addition(a, b):
2     """
3     Cette fonction prend deux nombres en entrée et renvoie leur somme.
4
5     :param a: Le premier nombre à additionner.
6     :type a: int ou float
7     :param b: Le deuxième nombre à additionner.
8     :type b: int ou float
9     :return: La somme des deux nombres.
10    :rtype: int ou float
11    """
12    return a + b
```

```
1 def addition(a, b):
2     """
3     Cette (a: Any, b: Any) -> Any
4
5     :para a: Le premier nombre à additionner.
6     :type Cette fonction prend deux nombres en entrée et renvoie leur
7     :para somme.
8     :type
9     :retu :param a: Le premier nombre à additionner.
10    :rtyp :type a: int ou float
11    :type :param b: Le deuxième nombre à additionner.
12    :type b: int ou float
13    :return: La somme des deux nombres.
14    :rtype: int ou float
15    """
16    return addition()
```

Documentation de fonctions

Documenter les fonctions en Python est une pratique essentielle pour améliorer la lisibilité et la maintenance du code. Les commentaires, sous forme de docstrings, sont placés juste après la signature de la fonction et fournissent des informations détaillées sur son utilisation, ses paramètres, ses valeurs de retour et d'autres détails importants.

```
1 def addition(a, b):
2     """
3     Cette fonction prend deux nombres en entrée et renvoie leur somme.
4
5     :param a: Le premier nombre à additionner.
6     :type a: int ou float
7     :param b: Le deuxième nombre à additionner.
8     :type b: int ou float
9     :return: La somme des deux nombres.
10    :rtype: int ou float
11    """
12    return a + b
```

```
1 def addition(a, b):
2     """
3     Cette (a: Any, b: Any) -> Any
4
5     :para a: Le premier nombre à additionner.
6     :type Cette fonction prend deux nombres en entrée et renvoie leur
7     :para somme.
8     :type
9     :retu :param a: Le premier nombre à additionner.
10    :rtyp :type a: int ou float
11    :param b: Le deuxième nombre à additionner.
12    :type b: int ou float
13    :return: La somme des deux nombres.
14    :rtype: int ou float
15    """
16    return addition()
```

Pratiques de programmation

Lors de la mise en place de fonctions, il est important de respecter certaines bonnes pratiques pour garantir la qualité et la maintenabilité du code.

- **Nommage explicite** : Choisissez des noms de fonction descriptifs et explicites qui reflètent clairement leur but et leur fonction dans le contexte de votre programme. Utilisez le style `snake_case` pour les noms de fonctions, en les séparant par des underscores.
- **Limitation des effets secondaires** : Évitez d'avoir des effets secondaires non intentionnels dans vos fonctions, comme la modification des variables globales. Privilégiez les fonctions qui prennent des arguments et renvoient des valeurs plutôt que de manipuler directement des variables globales.
- **Division en petites unités** : Décomposez les tâches complexes en fonctions plus petites et spécialisées, chaque fonction ayant une seule responsabilité. Cela rend votre code plus modulaire, facile à comprendre et à réutiliser.

Exercices

1. Écrivez une fonction nommée `calcul_moyenne()` qui prend une liste de notes en entrée et retourne la moyenne de ces notes.
2. Implémentez une fonction appelée `recherche_min()` qui prend une liste de nombres en entrée et retourne le plus petit de ces nombres.
3. Créez une fonction `generer_email()` qui prend un prénom et un nom en entrée, et retourne une adresse e-mail correspondante avec un domaine prédéfini.
4. Écrivez une fonction `compte_mots()` qui prend une chaîne de caractères représentant une phrase en entrée et retourne le nombre de mots dans cette phrase.
5. Écrivez une fonction `convertir_temperature()` qui prend une température en degrés Celsius et la convertit en degrés Fahrenheit.

Exercices

6. Créez une fonction `nombres_pairs_impairs()` qui prend une liste de nombres en entrée et retourne deux listes distinctes, l'une contenant les nombres pairs et l'autre les nombres impairs.
7. Implémentez une fonction `inverser_chaine()` qui prend une chaîne de caractères en entrée et retourne cette chaîne inversée.
8. Créez une fonction `valider_mot_de_passe()` qui prend un mot de passe en entrée et vérifie s'il répond à certains critères de complexité (longueur minimale, présence de chiffres, de lettres majuscules et minuscules, de caractères spéciaux, etc.). La fonction devrait renvoyer `True` si le mot de passe est valide et `False` sinon.

9. Gestions des erreurs

- Introduction aux exceptions
- Types d'exceptions
- Structure d'un bloc try-except
- Utilisation de plusieurs clauses except
- Bloc finally
- Gestion des exceptions spécifiques
- Gestion des erreurs avec contexte
- Astuces de débogage

Introduction aux exceptions

Les **exceptions** en Python sont des événements qui se produisent lorsqu'une erreur survient pendant l'exécution d'un programme.

Elles sont utilisées pour gérer les erreurs et les situations exceptionnelles.

Les **exceptions** jouent un rôle important dans la gestion des erreurs nous permettant de détecter, signaler et gérer les problèmes qui surviennent lors de l'exécution d'un programme.

Types d'exceptions

Voici un tableau présentant quelques-unes des exceptions intégrées couramment rencontrées :

Exception	Description
TypeError	Levée lorsqu'une opération est effectuée sur un objet d'un type inapproprié.
ValueError	Levée lorsqu'une opération est effectuée sur un objet avec une valeur inappropriée.
KeyError	Levée lorsqu'une clé inexistante est tentée d'être accédée dans un dictionnaire.
IndexError	Levée lorsqu'un index inexistant est tenté d'être accédé dans une séquence.
SyntaxError	Levée lorsqu'une syntaxe Python incorrecte est détectée.

<https://docs.python.org/fr/3/library/exceptions.html>

Types d'exceptions

AttributeError	Levée lorsqu'une référence d'attribut ou d'attribut invalide est utilisée.
NameError	Levée lorsqu'un nom de variable ou de fonction non défini est utilisé.
FileNotFoundError	Levée lorsqu'une tentative est faite pour ouvrir ou manipuler un fichier qui n'existe pas.
ZeroDivisionError	Levée lorsqu'une tentative de division par zéro est effectuée.
ImportError	Levée lorsqu'une erreur se produit lors de l'importation d'un module ou d'un package.
TypeError	Levée lorsqu'une opération est effectuée sur un objet d'un type inapproprié.
OSError	Levée lorsqu'une erreur système se produit (par exemple, lors de l'accès à un fichier).
AssertionError	Levée lorsqu'une instruction assert échoue.
RuntimeError	Levée lorsqu'une erreur se produit qui ne peut pas être attribuée à une catégorie spécifique d'exception.
StopIteration	Levée pour indiquer que la fin d'une séquence a été atteinte.

Structure d'un bloc try except

Nous pourrions par exemple réaliser une fonction division qui tente de diviser deux nombres fournis en arguments. Si une division par zéro est tentée, une exception de type `ZeroDivisionError` est levée. Cette exception est capturée par le bloc `except`, qui affiche un message approprié.

```
1  # Définition d'une fonction qui divise deux nombres
2  def division(a, b):
3      try:
4          result = a / b
5          print("Résultat de la division :", result)
6      except ZeroDivisionError:
7          print("Erreur : division par zéro")
8
9  # Appel de la fonction avec différents arguments
10 division(10, 2)  # Affichage: Résultat de la division : 5.0
11 division(10, 0)  # Affichage: Erreur : division par zéro
```

Utilisation de plusieurs clauses except

De la même manière qu'un bloc `if` peut être utilisé pour tester plusieurs conditions différentes, un bloc `try-except` peut également gérer plusieurs types d'erreurs distincts. Chaque clause `except` dans un bloc `try-except` spécifie un type d'exception particulier à gérer.

Ainsi, lorsqu'une exception se produit, Python parcourt chaque clause `except` dans l'ordre et exécute le bloc de code correspondant à la première clause dont le type d'exception correspond à l'erreur rencontrée.

```
1 # Définition d'une fonction qui convertit une chaîne en entier
2 def conversion_entier(chaine):
3     try:
4         nombre = int(chaine)
5         print("Conversion réussie :", nombre)
6     except ValueError:
7         print("Erreur : la chaîne ne peut pas être convertie en entier")
8     except TypeError:
9         print("Erreur : l'argument n'est pas une chaîne")
10
11 # Appel de la fonction avec différentes chaînes en argument
12 conversion_entier("123") # Affichage: Conversion réussie : 123
13 conversion_entier("abc") # Affichage: Erreur : la chaîne ne peut pas être convertie en entier
14 conversion_entier(123)   # Affichage: Erreur : l'argument n'est pas une chaîne
```

Bloc finally

De manière similaire à la gestion des exceptions avec plusieurs clauses `except`, le bloc `finally` offre une flexibilité supplémentaire dans la gestion des erreurs en Python.

Le bloc `finally` permet d'exécuter un code spécifique après l'exécution du bloc `try` et de toutes les clauses `except`, quel que soit le résultat de l'opération.

Cela signifie que le code à l'intérieur du bloc `finally` sera toujours exécuté, que des exceptions se produisent ou non.

Cette fonctionnalité est souvent utilisée pour effectuer des opérations de nettoyage ou de fermeture, garantissant que certaines actions critiques sont toujours effectuées, même en cas d'erreur.

Bloc finally

Ici, nous tentons d'ouvrir un fichier "mon_fichier.txt" en lecture. Si le fichier n'est pas trouvé, une exception `FileNotFoundError` est levée, et le message "Le fichier n'a pas été trouvé." est affiché.

Peu importe le résultat, le bloc `finally` est exécuté après le bloc `try` et toutes les clauses `except`. Dans ce cas, le bloc `finally` ferme le fichier avec la méthode `close()` pour nettoyer les ressources.

```
1  try:
2      fichier = open("mon_fichier.txt", "r")
3      contenu = fichier.read()
4      print(contenu)
5  except FileNotFoundError:
6      print("Le fichier n'a pas été trouvé.")
7  finally:
8      if 'fichier' in locals():
9          fichier.close()
10         print("Fermeture du fichier.")
```

Gestions des exceptions spécifiques

La gestion des exceptions spécifiques en Python consiste à traiter des types d'erreurs spécifiques de manière distincte et ciblée.

Plutôt que de capturer toutes les exceptions avec une seule clause `except`, on peut utiliser des clauses `except` individuelles pour différents types d'exceptions.

Cela permet de personnaliser la réponse à chaque type d'erreur, en prenant des mesures appropriées pour chaque situation.

```
1  def operation(a, b):
2      if not isinstance(a, int) or not isinstance(b, int):
3          raise TypeError("Les arguments doivent être des entiers.")
4      if b == 0:
5          raise ValueError("Le deuxième argument ne peut pas être zéro.")
6      resultat_division = a / b
7      return resultat_division
8
9  try:
10     print(operation(10, 2))
11     print(operation("abc", 2))
12     print(operation(5, 0))
13 except (TypeError, ValueError) as e:
14     print("Erreur :", e)
```

Gestion des erreurs avec contexte

Le bloc `with` en Python est utilisé pour garantir une gestion efficace des ressources, telles que les fichiers ou les connexions réseau. Lorsque vous utilisez un bloc `with`, vous pouvez être sûr que les ressources seront libérées correctement, même en cas d'erreur.

Par exemple, lors de l'ouverture d'un fichier, le bloc `with` s'assure que le fichier est fermé automatiquement une fois que vous avez fini de l'utiliser, même si une exception se produit pendant l'exécution du code à l'intérieur du bloc `with`. Cela garantit une utilisation sûre et efficace des ressources système.

```
1  # Ouverture d'un fichier en mode écriture avec le bloc with
2  with open("mon_fichier.txt", "w") as fichier:
3      fichier.write("Contenu du fichier écrit avec Python\n")
4      fichier.write("Deuxième ligne\n")
```

Astuces de débogage

Pour déboguer efficacement les erreurs en Python, plusieurs techniques sont disponibles.

La fonction **traceback** est particulièrement utile pour tracer les erreurs jusqu'à leur origine, offrant ainsi une visibilité complète sur le parcours de l'exécution du programme.

En outre, la journalisation est une pratique courante pour enregistrer des informations détaillées sur les erreurs, telles que leur cause et leur contexte d'apparition.

```
1 import traceback
2 def division(a, b):
3     try:
4         return a / b
5     except ZeroDivisionError as e:
6         traceback.print_exc()
7 division(5, 0)
```

PROBLEMS 1 OUTPUT PORTS SERIAL MONITOR SQL CONSOLE TERMINAL ...

Microsoft Windows [version 10.0.19045.4046]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\Caven\Desktop\demo>C:/Users/Caven/AppData/Local/Programs/Python/
/Caven/Desktop/demo/traceback_1.py
Traceback (most recent call last):
File "c:\Users\Caven\Desktop\demo\traceback_1.py", line 5, in division
 return a / b
~~~~~  
ZeroDivisionError: division by zero

**Merci pour votre attention.**